

CHAOS ARENA 2

Documentacion tecnica del codigo fuente

Android fighting game - Java / Android SDK

Namespace : riberadeltajo.es.chaosarena2

Paquetes : engine | entity | ai | input | stage | progression

Clases : 13 ficheros Java

Plataforma: Android (minSdk 24, targetSdk 34)

Índice de clases

ActividadJuego	riberadeltajo.es.chaosarena2	Actividad Android principal (punto de entrada)
BucleJuego	riberadeltajo.es.chaosarena2.engine	Hilo del bucle de juego a 60 FPS
Juego	riberadeltajo.es.chaosarena2.engine	Vista principal: logica, pantallas y renderizado
ResourceManager	riberadeltajo.es.chaosarena2.engine	Carga y centraliza assets visuales
Player	riberadeltajo.es.chaosarena2.entity	Entidad jugador/enemigo: fisica, stats, animacion, hitbox
SpriteAnimation	riberadeltajo.es.chaosarena2.entity	Animacion por frames desde assets PNG
EnemyAI	riberadeltajo.es.chaosarena2.ai	Maquina de estados que controla al enemigo
VirtualJoystick	riberadeltajo.es.chaosarena2.input	Joystick tactil dibujado en Canvas
StageDef	riberadeltajo.es.chaosarena2.stage	Datos de un escenario de combate
Buff	riberadeltajo.es.chaosarena2.progression	Mejora aplicable al jugador en modo Arcade
BuffType	riberadeltajo.es.chaosarena2.progression	Enum de tipos de mejora disponibles
Event	riberadeltajo.es.chaosarena2.progression	Evento especial aleatorio entre combates
RunManager	riberadeltajo.es.chaosarena2.progression	Gestor del estado completo de una partida Arcade

Paquete: riberadeltajo.es.chaosarena2 (raiz)**class ActividadJuego**

riberadeltajo.es.chaosarena2

Punto de entrada de la aplicación Android. Extiende AppCompatActivity y gestiona el ciclo de vida del sistema operativo (onCreate/onPause/onResume/onDestroy). Además centraliza todas las constantes de configuración del juego como campos static públicos accesibles desde Juego.java sin referencias circulares entre paquetes.

Constantes estáticas (configuración global)

```
static final int MAX_LEVELS = 3
```

-> Número de niveles que tiene el modo Historia. Juego comprueba `currentLevel > MAX_LEVELS` para detectar la victoria final.

```
static final int MAX_SLOTS = 3
```

-> Número de ranuras de guardado. Reservado para una futura pantalla de selección de partida.

```
static final float[] LEVEL_ENEMY_HEALTH = {250, 250, 250}
```

-> Vida máxima del enemigo en cada nivel del modo Historia. Se asigna a `player2.maxHealth` en `applyModeConfig()`.

```
static final float[] LEVEL_ENEMY_SPEED = {300, 400, 500}
```

-> Velocidad de movimiento del enemigo en px/s (coordenadas mundo 1920x1080). Aumenta con cada nivel para hacer al enemigo progresivamente más agresivo.

```
static final float[] LEVEL_ENEMY_DAMAGE = {5, 5, 5}
```

-> Dano base del enemigo. Se pasa a `EnemyAI.setDamage()`; internamente la IA actualiza `player2.damageMultiplier = damage/10`.

```
static final float[] LEVEL_ATTACK_RATE = {1.6, 1.0, 0.6}
```

-> Segundos mínimos entre ataques del enemigo por nivel. Valor menor = más ataques por segundo. Nivel 1: ataque cada ~1.6s. Nivel 3: ataque cada ~0.6s.

```
static final String[] LEVEL_ENEMY_NAMES
```

-> Nombres de los rivales en modo Historia: 'Dark Fighter' (nv1), 'Ronin Boss' (nv2), 'Shadow Master' (nv3). Usados en `resolveEnemyName()` de Juego para determinar el sprite.

```
static final String[] LEVEL_LORE
```

-> Array de 3 cadenas largas con el texto narrativo mostrado en la pantalla LORE antes de cada combate. Se indexa con `Math.min(currentLevel-1, length-1)` para evitar desbordamiento.

Campos de instancia

```
Juego juego
```

-> Vista principal del juego (SurfaceView). Creada en onCreate y destruida en onDestroy. Toda la lógica del juego reside en esta clase.

```
ResourceManager resources
```

-> Cargador de assets. Se instancia antes que Juego para que los assets estén disponibles cuando Juego los necesite en el constructor.

```
SharedPreferences prefs
```

-> Archivo de preferencias 'ChaosArenaPrefs' con modo `MODE_PRIVATE`. Almacena el top-5 de puntuaciones arcade y el estado de silencio de la música.

Métodos del ciclo de vida Android

```
onCreate(Bundle savedInstanceState)
```

-> Configura la actividad en pantalla completa sin barra de título: `requestWindowFeature(FEATURE_NO_TITLE)`, `FLAG_FULLSCREEN` y `FLAG_KEEP_SCREEN_ON`. Luego instancia `ResourceManager(this)`, `Juego(this, resources, prefs)` y llama a `setContentView(juego)` para ocupar toda la pantalla.

```
onPause()
```

-> Llama a `juego.pauseMusic()` para silenciar la música cuando la app pasa a segundo plano. Evita consumir CPU/batería con la reproducción de audio.

```
onResume()
```

-> Llama a `juego.resumeMusic()` para reanudar la música al volver a primer plano.

`onDestroy()`

-> Libera todos los recursos: `juego.releaseMusic()` libera el `MediaPlayer` y `resources.recycle()` recicla todos los `Bitmaps` de `assets` para evitar `memory leaks`.

Paquete: riberadeltajo.es.chaosarena2.engine**class extends Thread BucleJuego**

riberadeltajo.es.chaosarena2.engine

Hilo dedicado al bucle de juego que mantiene 60 FPS. Implementa un esquema de tiempo fijo (fixed timestep) con catch-up: si un frame tarda mas de 16ms, ejecuta hasta MAX_FRAMES_SALTADOS actualizaciones logicas extra sin renderizar para recuperar el retraso acumulado, manteniendo la simulacion fisica consistente independientemente de la carga del dispositivo.

Constantes

```
static final int MAX_FPS = 60
```

-> Frecuencia de actualizacion objetivo. Todos los calculos de tiempo se derivan de este valor.

```
static final int MAX_FRAMES_SALTADOS = 5
```

-> Maximo de actualizaciones logicas adicionales permitidas en un solo frame. Limita el efecto 'spiral of death' cuando el dispositivo es demasiado lento para mantener 60 FPS.

```
static final int TIEMPO_FRAME = 1000 / MAX_FPS (= 16 ms)
```

-> Duracion objetivo de cada frame en milisegundos. Se usa para calcular tiempoDormir.

Campos publicos

```
volatile boolean enEjecucion
```

-> Flag de control del bucle. La palabra clave volatile garantiza visibilidad entre hilos. El hilo principal lo pone a false en surfaceDestroyed() para parar el bucle limpiamente.

```
int iteraciones
```

-> Contador de frames completamente renderizados desde el inicio del bucle. Se puede usar para calcular FPS promedio: $\text{fps} = \text{iteraciones} / (\text{tiempoTotal} / 1000f)$.

```
long tiempoTotal
```

-> Suma de tiempos de todos los frames en ms. Permite diagnosticar el rendimiento global.

```
int maxX, maxY
```

-> Dimensiones reales de la pantalla en pixeles. Se leen del primer Canvas bloqueado en el constructor. Actualmente no se usan externamente pero sirven para debug.

Constructores y métodos

```
BucleJuego(SurfaceHolder sh, Juego juego)
```

-> Guarda referencias al SurfaceHolder y a Juego. Intenta bloquear el Canvas una vez para leer las dimensiones reales (maxX, maxY) y lo libera inmediatamente sin dibujar nada. El paquete-privado del constructor fuerza que solo Juego lo pueda instanciar.

```
run()
```

-> Bucle principal: (1) bloquea el Canvas del SurfaceHolder, (2) mide tiempoComienzo, (3) llama a juego.actualizar(delta) donde $\text{delta} = \text{TIEMPO_FRAME} / 1000f = 0.01667s$ fijo, (4) llama a juego.renderizar(canvas), (5) incrementa iteraciones, (6) calcula tiempoDormir = $\text{TIEMPO_FRAME} - \text{tiempoRealTranscurrido}$, (7) si tiempoDormir > 0 duerme ese tiempo para no quemar CPU, (8) si tiempoDormir < 0 ejecuta bucle catch-up: repite actualizar(delta) hasta MAX_FRAMES_SALTADOS veces sin renderizar para ponerse al dia, (9) en el bloque finally libera siempre el Canvas.

class extends SurfaceView implements SurfaceHolder.Callback Juego

[iberadeltajo.es/chaosarena2.engine](https://github.com/iberadeltajo.es/chaosarena2.engine)

Clase central que actúa como vista Android y como controlador de toda la lógica. Implementa una máquina de estados con 8 pantallas (Screen enum). Escala el mundo virtual 1920x1080 a cualquier resolución real mediante una Matrix de transformación. Gestiona jugadores, IA, música, entrada táctil multi-punto y la progresión arcade completa.

Constantes de diseño

```
float WORLD_W = 1920, WORLD_H = 1080
```

-> Resolución del espacio de juego virtual. Toda la lógica opera en estas coordenadas. worldMatrix las escala al renderizar para adaptarse a cualquier pantalla.

```
float INITIAL_TIME = 90
```

-> Segundos por combate en modo Arcade. Cuando llega a 0 el jugador pierde.

```
float MOVE_SPEED = 650
```

-> Velocidad horizontal base del jugador en px/s (coordenadas mundo). Se multiplica por player1.speedMultiplier en updateFighting().

```
float MAP_MIN_X = 150
```

-> Límite horizontal: los personajes no pueden salir del rango [150, WORLD_W-150].

```
float DEATH_ANIM_WAIT = 2.5
```

-> Segundos de espera tras activar pendingResult antes de mostrar la pantalla de resultado. Permite que se vea completa la animación de muerte del personaje derrotado.

```
float HIT_FLASH_DURATION = 0.10
```

-> Duración del círculo de impacto amarillo en el punto de golpe. 100ms de efecto visual.

Máquina de estados

```
enum Screen { MAIN_MENU, CHAR_SELECT, LORE, FIGHTING, PAUSED, RESULT, BUFF_SELECT, EVENT }
```

-> MAIN_MENU: pantalla de inicio con 3 modos. CHAR_SELECT: elección de personaje y escenario. LORE: texto narrativo antes del combate (solo Historia). FIGHTING: combate activo. PAUSED: pausa superpuesta sobre el combate. RESULT: resultado con opciones de continuar/menú. BUFF_SELECT: elección de mejora (solo Arcade). EVENT: evento especial aleatorio (solo Arcade).

Campos de estado del combate

```
Player player1, player2
```

-> Jugador humano (player1) y oponente controlado por la IA o segundo jugador humano (player2). Se crean en startFight() y se reutilizan entre combates via spawnNewEnemy().

```
EnemyAI enemyAI
```

-> Controlador de IA del enemigo. Su update(delta) se llama cada frame en updateFighting(). Solo controla a player2 en modos Historia y Duelo.

```
RunManager runManager
```

-> Instanciado solo en modo Arcade (activeSlot == -2). Gestiona buffs, eventos y escalado progresivo de enemigos. null en Historia y Duelo.

```
int activeSlot
```

-> Modo activo: >=0 Historia (índice de slot de guardado), -1 Duelo local, -2 Arcade.

```
int currentLevel
```

-> Nivel actual en modo Historia (1..MAX_LEVELS). Incrementado en resetCombat() tras victoria.

```
boolean pendingResult, pendingResultWon
```

-> pendingResult=true indica que se está esperando que transcurra DEATH_ANIM_WAIT. pendingResultWon guarda si el jugador gana o pierde para pasarlo a showResult().

```
float deathDelay
```

-> Acumulador de tiempo (en segundos) desde que se activo pendingResult. Cuando supera DEATH_ANIM_WAIT se llama a showResult().

StageDef currentStage

-> Escenario activo durante el combate. Apunta a uno de los 3 elementos de res.stages[].

Escalado y matriz**Matrix worldMatrix / inverseMatrix**

-> worldMatrix: setScale(screenW/WORLD_W, screenH/WORLD_H). Se aplica con canvas.concat(worldMatrix) al inicio de renderizar(). inverseMatrix: inversa calculada con worldMatrix.invert(inverseMatrix). Se usa para convertir coordenadas de toque de pantalla a coordenadas mundo.

VirtualJoystick joystick

-> Joystick virtual en coordenadas mundo (centerX=280, centerY=WORLD_H-280=800, radio=180). Creado en el constructor y actualizado con la inverseMatrix en updateScaleMatrix().

float hitFlashTimer, hitFlashX, hitFlashY

-> Estado del efecto de impacto. hitFlashTimer cuenta desde HIT_FLASH_DURATION hacia 0. Mientras > 0 se dibuja un círculo amarillo en (hitFlashX, hitFlashY) con radio y opacidad proporcionales al tiempo restante.

Métodos de ciclo de vida (SurfaceHolder.Callback)**surfaceCreated(SurfaceHolder holder)**

-> Bloquea el Canvas para obtener dimensiones reales, llama a updateScaleMatrix(), crea e inicia BucleJuego y arranca la música con startMusic().

surfaceChanged(int format, int w, int h)

-> Recalcula worldMatrix e inverseMatrix con las nuevas dimensiones. También actualiza la inverseMatrix del joystick.

surfaceDestroyed(SurfaceHolder holder)

-> Para el hilo BucleJuego: pone enEjecucion=false y hace join() en bucle hasta que el hilo termine limpiamente.

Bucle principal**actualizar(float delta)**

-> Delegado de BucleJuego. Solo llama a updateFighting(delta) si currentScreen==FIGHTING.

renderizar(Canvas canvas)

-> Aplica worldMatrix al canvas y despacha al método draw* según currentScreen. PAUSED y RESULT dibujan primero drawFighting() y luego superponen su overlay.

Inicio de combate**updateFighting(float delta)**

-> Núcleo del combate: (1) si pendingResult acumula deathDelay hasta DEATH_ANIM_WAIT, actualiza ambos jugadores y transiciona a showResult; (2) en Arcade decrementa timeLeft y activa derrota si llega a 0; (3) lee joystick para mover/rotar player1; (4) llama a player1.update/updateAttack y player2.update/updateAttack; (5) llama a enemyAI.update; (6) procesa justHitThisFrame para disparar hitFlash; (7) clipa posición X de ambos en [MAP_MIN_X, WORLD_W-MAP_MIN_X]; (8) comprueba currentHealth <= 0 y llama a triggerResult().

triggerResult(boolean won)

-> Activa pendingResult: llama a player2.startDeath() si el jugador gana o player1.startDeath() si el jugador perdió. El otro personaje queda en forceIdle(). Evita activarse dos veces con if(pendingResult) return.

showResult(boolean won)

-> Guarda el resultado en lastBattleWon y transiciona a Screen.RESULT. En Arcade: si won, llama a runManager.onFightWon() e incrementa enemiesDefeated; si no won, llama a saveArcadeScore().

Inicio de pantalla y progresión**startFight()**

-> Inicializa el combate: resuelve nombre de personaje del jugador y del enemigo, crea ambos Player con sus sprites y posiciones iniciales, los enlaza (opponent), crea EnemyAI, llama a applyModeConfig(), crea RunManager en Arcade. Transiciona a LORE (Historia) o FIGHTING (Arcade/Duelo).

spawnNewEnemy()

-> Reutiliza el objeto player2 cargando nuevas animaciones con loadAnimations(). Evita crear y destruir objetos Player entre rondas en Arcade e Historia.

applyModeConfig()

-> Configura los stats del enemigo según el modo: Historia usa LEVEL_ENEMY_* de ActividadJuego; Arcade fija valores base (250 vida, 350 speed, 10 dmg, 1.5s rate); Duelo da 600 de vida a ambos y IA más agresiva.

`onResultAction()`

-> Botón de acción de la pantalla RESULT: Arcade+victoria -> BUFF_SELECT; Arcade+derrota -> MAIN_MENU; Historia/Duelo -> resetCombat().

`checkForEvent()`

-> Si `runManager.shouldTriggerEvent()`: genera evento, lo aplica y va a EVENT. Si no: llama a `resetCombat()`.

`resetCombat()`

-> Reinicia posiciones (`player1.x=400`, `player2.x=WORLD_W-400`), vidas, velocidades y `comboCharge`. En Historia avanza el nivel. En Arcade llama a `runManager.applyEnemyScaling()` para escalar al nuevo enemigo.

`saveArcadeScore()`

-> Lee el top-5 de `SharedPreferences`, agrega la nueva puntuación al `array[6]`, ordena por tiempo de supervivencia (burbuja) y guarda de vuelta los 5 primeros.

Entrada de teclados

`onTouchEvent(MotionEvent e)` -> boolean

-> En combate/pausa: redirige el evento al joystick para procesarlo. Para ACTION_DOWN y ACTION_POINTER_DOWN: convierte las coordenadas del puntero activo de pantalla a mundo con `inverseMatrix.mapPoints()` y llama a `handleTap(wx, wy)`.

`handleTap(float wx, float wy)`

-> Switch sobre `currentScreen`. Usa `hit(RectF, x, y)` para cada botón de la pantalla activa. Cubre los 8 estados: MAIN_MENU (3 modos), CHAR_SELECT (personaje+escenario+luchar), LORE (empezar), FIGHTING (pausa+4 ataques), PAUSED (continuar+salir), RESULT (acción+menu), BUFF_SELECT (3 mejoras), EVENT (continuar).

Música y dibujo

`startMusic()`

-> Abre 'sounds/Techno_Syndrome.mp3' desde assets con `AssetFileDescriptor`, configura el `MediaPlayer` en loop y ajusta el volumen según la preferencia `is_muted`.

`drawFighting(Canvas canvas)`

-> Dibuja: fondo con desplazamiento parallax horizontal (`pShiftX` basado en `player1.x`), `player1.draw()` y `player2.draw()`, círculo de impacto amarillo si `hitFlashTimer>0`, overlay rojo semi-transparente si `player1` usa SPECIAL, barras de vida/combo y botones.

`initAllButtonRects()`

-> Pre-calcula posiciones de todos los botones de todas las pantallas una sola vez en el constructor. Los botones son `RectF` en coordenadas mundo que se comparan directamente con las coordenadas de toque convertidas.

class ResourceManager

[iberadeltajo.es/chaosarena2.engine](https://github.com/iberadeltajo.es/chaosarena2.engine)

Unico punto de carga de assets visuales. Se instancia antes de Juego y expone todos los recursos como campos publicos finales para acceso directo sin recargas. Proporciona helpers para resolver nombres de personaje a rutas de sprites.

Assets de fuente

Paint fontSmall (~36sp, blanco, BOLD)

-> Para textos de HUD: nombres de personaje, enemigos derrotados, textos pequenos en botones.

Paint fontMedium (~52sp, blanco, BOLD)

-> Para etiquetas de botones de accion en todas las pantallas.

Paint fontBig (~72sp, blanco, BOLD)

-> Para titulos principales, 'CHAOS ARENA', '¡VICTORIA!', 'DERROTA', contador de tiempo.

Assets de fondo

Bitmap bgTitle

-> Fondo del menu principal. Cargado desde backgrounds/title_bg.png. Se dibuja escalado a WORLD_W x WORLD_H.

Bitmap joystickBg, joystickKnob

-> Imagenes del joystick virtual: joystick/AIR_joystick_bg600.png (fondo circular) y AIR_joystick_stick600.png (bola movil). Dibujadas por VirtualJoystick.draw().

Bitmap soundOn, soundOff

-> Iconos del boton de silencio (ui/sound_on.png, ui/sound_off.png). Reservados para la HUD, actualmente no se muestran en pantalla.

StageDef[] stages (3 elementos)

-> Array de escenarios: [0] Bosque (forest_bg, floorOffset=-40, groundY=80), [1] Desierto (desert_bg, offset=0, groundY=60), [2] Ciudad (city_bg, offset=40, groundY=90).

AssetManager assets

-> Referencia al AssetManager de Android. Se pasa a Player.loadAnimations() para que pueda abrir los PNG de los sprites en tiempo de ejecucion.

Métodos

ResourceManager(Context context)

-> Obtiene assets=context.getAssets(), crea los tres Paint (makePaint), carga los Bitmaps de fondo e iconos (loadBitmap) y construye el array stages con sus fondos.

getFolderForChar(String name) -> String

-> Mapea nombre a carpeta de assets: contiene 'Goro' -> 'sprites/goro', contiene 'Sub Zero' -> 'sprites/subzero', resto -> 'sprites/liu_kang'.

getPrefixForChar(String name) -> String

-> Mapea nombre a prefijo de frames: 'goro', 'subzero' o 'liu_kang'. Se concatena con '_idle', '_run', '_attack1', etc. para buscar los PNGs.

loadBitmap(String assetPath) -> Bitmap

-> Decodifica un asset con BitmapFactory.decodeStream(inScaled=false). Si el fichero no existe devuelve Bitmap 1x1 magenta como indicador visual de error.

recycle()

-> Llama a recycleBitmap() en bgTitle, joystickBg, joystickKnob, soundOn, soundOff y en cada StageDef del array stages.

Paquete: riberadelta.es.chaosarena2.entity**class Player**`riberadelta.es.chaosarena2.entity`

Clase mas compleja del juego. Representa tanto al jugador humano como al enemigo. Contiene fisica (gravedad, salto), animacion por estados, sistema de hitboxes en coordenadas canvas, stats modificables por buffs, y renderizado con espejo horizontal. El sistema de coordenadas usa Y-arriba para la fisica ($y=0$ en el suelo) y convierte a Y-abajo de Canvas solo en draw() con la formula: $\text{drawY} = (\text{worldHeight} - y) + \text{drawOffsetY}$.

```
enum { AttackType: PUNCH, KICK, SPECIAL }
```

-> Tipo de ataque activo. PUNCH: golpe basico (10 dmg, alcance 300px). KICK: patada (15 dmg, alcance 400px). SPECIAL: combo devastador (45 dmg, alcance 700px, requiere comboCharge ≥ 100).

```
enum { State: IDLE, WALK, ATTACKING, HURT, DEAD }
```

-> Estado del personaje. Determina la animacion reproducida y las acciones permitidas. HURT y DEAD bloquean movimiento y ataques. ATTACKING solo acepta multiples golpes cuando hasHitInCurrentAttack es false.

```
float x, y
```

-> Posicion en coordenadas mundo. x: horizontal (0=izquierda, 1920=derecha). y: vertical con origen en el suelo y Y aumentando hacia arriba. Convertido a canvas en draw(): $\text{drawY} = (\text{worldHeight} - y) + \text{drawOffsetY}$.

```
float velocityY
```

-> Velocidad vertical en px/s. Se reduce cada frame: $\text{velocityY} += \text{GRAVITY} * \text{delta}$. Al saltar se asigna $\text{velocityY} = \text{JUMP_FORCE}$.

```
static final float GRAVITY = -2500
```

-> Aceleracion gravitacional en px/s^2 . Negativo porque el eje Y sube en la fisica. Con JUMP_FORCE=1150, el personaje tarda ~0.46s en llegar al punto mas alto del salto.

```
static final float JUMP_FORCE = 1150
```

-> Velocidad vertical inicial al saltar. Produce un salto con altura maxima de ~264px.

```
float groundY
```

-> Coordenada Y del suelo para este personaje especifico. Inicializada con el parametro y del constructor y se usa como limite inferior en update().

```
float scale = 4.0
```

-> Factor de escala aplicado a todos los frames de sprite al dibujar. Un frame de 47x94px (Liu Kang idle) se dibuja como 188x376px en pantalla.

```
float drawOffsetX = 0, drawOffsetY
```

-> Offset en pixeles canvas aplicado DESPUES de convertir Y-fisica a Y-canvas. drawOffsetY es negativo para desplazar el sprite hacia arriba y alinear los pies del sprite con groundY. Valores: subzero=-636, goro=-472, liu_kang=-396.

```
float hbwMult, hbhMult
```

-> Multiplicadores sobre el tamaño del frame para calcular dimensiones de la hitbox. subzero: $w=0.105$ (~84px), $h=0.490$ (~374px). goro y liu_kang: $w=0.35$, $h=0.80$.

```
float hbOffsetX, hbOffsetY
```

-> Offset de posicion de la hitbox en pixeles canvas respecto al centro del frame. Necesario cuando el contenido del sprite no esta centrado en su frame. subzero: $\text{hbOffsetX}=-58$ (personaje centrado en $X=85$ de un frame de 199px ancho, $\text{delta}=(99.5-85)*4=58$), $\text{hbOffsetY}=164$ (41px vacios arriba del frame * 4 = 164px).

```
float maxHealth = 250, currentHealth = 250
```

-> Vida del personaje. currentHealth llega a 0 cuando muere. maxHealth puede aumentar con buffs HEALTH o con el escalado de RunManager en enemigos.

```
float comboCharge, MAX_COMBO_CHARGE = 100
```

-> Barra de ataque especial. Aumenta con cada golpe de PUNCH o KICK aplicado mediante addCharge(dmg). Se consume a 0 al usar SPECIAL.

```
float damageMultiplier = 1.0
```

-> Multiplicador aplicado al dano de todos los ataques. Aumentado por buffs DAMAGE y por el escalado de RunManager en enemigos.

```
float speedMultiplier = 1.0
```

-> Multiplicador de MOVE_SPEED aplicado en Juego.updateFighting(). Aumentado por buffs SPEED.

```
float lifestealPercent = 0
```

-> Porcentaje (0..1) del dano infligido que se recupera como vida. Ej: 0.05 -> 5%. Se aplica despues de calcular el dano real en updateAttack().

```
float armorPercent = 0
```

-> Porcentaje de reduccion del dano recibido. Ej: 0.10 -> 10% de reduccion. Se aplica en takeDamage(): reduced = amount * max(0, 1 - armorPercent).

```
float criticalChance = 0
```

-> Probabilidad (0..1) de golpe critico (dano x2). Se evalua con Math.random() en updateAttack() antes de aplicar el dano.

```
float regenPerSec = 0
```

-> HP recuperados por segundo de forma pasiva. Se aplica en update() si currentHealth > 0 y currentHealth < maxHealth.

```
boolean facingRight
```

-> true si el personaje mira hacia la derecha. false invierte el sprite horizontalmente en draw() mediante Matrix.setScale(-1, 1).

```
Player opponent
```

-> Referencia al adversario. Usada en takeDamage() para calcular la direccion del knockback y en canHit() para detectar colision de hitboxes.

```
boolean hasHitInCurrentAttack
```

-> Evita multiples golpes en el mismo swing. Se resetea a false al iniciar y terminar un ataque en updateAttack().

```
boolean justHitThisFrame
```

-> true exactamente durante un frame cuando se produce un golpe. Juego.triggerHitFlash() lo lee para disparar el efecto visual de impacto.

Flags de animacion

```
boolean hasJumpAnim, hasKickAnim, hasDeathAnim, hasHurtAnim
```

-> Indican si el personaje tiene animacion propia para cada accion. Se detectan comprobando getFrameWidth() > 1 (la animacion fallback tiene 1px de ancho). Si no existe, currentFrame() usa animaciones alternativas.

Debug visual

```
static boolean DEBUG_HITBOXES = true
```

-> Si true, draw() superpone un rectangulo verde (hitbox de cuerpo) y uno rojo (zona de alcance del ataque) sobre cada personaje. Cambiar a false para release.

```
Paint dbgBodyPaint (estatico, verde 0xFF00FF00, STROKE, grosor=4)
```

-> Paint para dibujar el borde verde de la hitbox de cuerpo.

```
Paint dbgAttackPaint (estatico, rojo 0xFFFF3333, STROKE, grosor=4)
```

-> Paint para dibujar el borde rojo de la zona de alcance del ataque activo.

Métodos de constructor

```
Player(name, assets, folder, charPrefix, x, y, facingRight, worldHeight)
```

-> Constructor. Configura hurtPaint con filtro PorterDuff rojo semi-transparente (0xA AFF0000, SRC_ATOP) para el efecto de parpadeo al recibir dano. Llama a loadAnimations().

```
loadAnimations(AssetManager assets, String folder, String prefix, float worldHeight)
```

-> Carga las 7 animaciones: idle(0.12s LOOP), run(0.10s LOOP), attack1(0.08s NORMAL), attack2(0.10s NORMAL),

takehit(0.12s NORMAL), jump(0.10s LOOP), death(0.12s NORMAL). Detecta existencia comprobando frameWidth>1. kickVisualAnim = kickAnim si existe, o 'run' a 0.07s NORMAL como visual alternativo. Llama a calibrateScale().

calibrateScale(String prefix)

-> Ajusta los 6 parametros de escala/hitbox segun el personaje: subzero: scale=4, drawOffsetY=-636, hbWMult=0.105, hbHMult=0.490, hbOffsetX=-58, hbOffsetY=164. Calculo: frame 199x191px, contenido real en X=57..113 (centro=85), Y=41..159. Centro del frame=99.5 -> delta=(99.5-85)*4=58px canvas; vacios arriba=41*4=164px. goro: hasHurtAnim=false (animacion takehit tiene caida no deseada). liu_kang: valores por defecto (hbWMult=0.35, hbHMult=0.80).

Lógica de actualización

update(float delta)

-> Gestiona la simulacion: si DEAD solo avanza deathTime y retorna. Si no: velocityY += GRAVITY*delta, y += velocityY*delta, clipa en groundY. Si HURT: avanza hurtTime, vuelve a IDLE si hurtAnim.isFinished(hurtTime). Aplica regeneracion si regenPerSec>0. Siempre avanza stateTime y jumpTime.

updateAttack(float delta)

-> Resetea justHitThisFrame. Si no ATTACKING retorna. Avanza attackTime. Calcula totalDuration segun AttackType (SPECIAL = punchDuration + kickDuration). Si attackTime >= totalDuration: fin del ataque, resetea estado. Si attackTime > hitTime y !hasHitInCurrentAttack y canHit(opponent): calcula dano (baseDmg * damageMultiplier, x2 si critico), aplica knockback, aplica lifesteal, marca hasHitInCurrentAttack=true y justHitThisFrame=true.

move(float amount)

-> Si no ATTACKING ni HURT: desplaza x en amount px y pone estado WALK (si esta en suelo).

updateDirection(boolean r, boolean l)

-> Actualiza facingRight segun entradas. Si ninguna: si WALK pone IDLE.

attack(AttackType type)

-> Si no ATTACKING ni HURT: SPECIAL requiere comboCharge>=100. Inicializa estado, tipo, attackTime=0, hasHitInCurrentAttack=false. SPECIAL consume comboCharge=0.

takeDamage(float amount, float knockback)

-> Calcula dano reducido por armadura, resta de currentHealth (minimo 0), pone estado HURT, resetea hurtTime=0. Knockback: pushDir = +1 si opponent esta a la izquierda, -1 si esta a la derecha. x += knockback * pushDir.

jump()

-> Si isGrounded() y no HURT: velocityY=JUMP_FORCE, jumpTime=0.

forceIdle()

-> Fuerza State.IDLE y resetea attackTime y hurtTime. Usado en resetCombat().

startDeath()

-> Pone State.DEAD, deathTime=0, velocityY=0. Llamado por Juego.triggerResult().

Hitbox y detección de colisión

getHitbox() -> RectF

-> Calcula el rectangulo de cuerpo en coordenadas canvas: hbW = frameWidth * scale * hbWMult; hbH = frameHeight * scale * hbHMult. ox = facingRight ? hbOffsetX : -hbOffsetX (inverso si mira a la izquierda). cx = x + ox. top = (worldHeight - y) + drawOffsetY + hbOffsetY. Retorna RectF(cx-hbW/2, top, cx+hbW/2, top+hbH).

canHit(Player other) -> boolean

-> Requiere other!=null y State.ATTACKING. Calcula reach segun tipo: KICK=400, SPECIAL=700, PUNCH=300. ax = facingRight ? body.right : body.left - reach. Retorna RectF.intersects(area, other.getHitbox()).

Dibujos

draw(Canvas canvas)

-> drawX = x - frameWidth*scale/2 + drawOffsetX. drawY = (worldHeight - y) + drawOffsetY. Si !facingRight: matrix.setScale(-1,1) + postTranslate(frameWidth,0) para espejo. matrix.postScale(scale,scale) + postTranslate(drawX, drawY). Dibuja con hurtPaint si State.HURT y (hurtTime*15)%2==0 (parpadeo cada 2 ticks de 15Hz). Si DEBUG_HITBOXES llama a drawDebugHitbox().

getBodyCanvasY() -> float

-> $\text{top} + \text{frameHeight} * \text{scale} * 0.35f$ (35% del alto = nivel del torso). Usada por `Juego.triggerHitFlash()` como punto de impacto del efecto visual.

`currentFrame()` -> Bitmap

-> Selector de frame según estado y timers: DEAD -> `deathAnim(deathTime)` o último frame de `hurtAnim` si no hay death. HURT -> `hurtAnim(hurtTime)` o `idle[0]`. ATTACKING SPECIAL -> `punchAnim` primera mitad, `kickVisualAnim` segunda mitad. ATTACKING KICK -> `kickVisualAnim`. ATTACKING PUNCH -> `punchAnim`. En aire (`!isGrounded`) -> `jumpAnim`. WALK -> `walkAnim`. Default -> `idleAnim`.

class SpriteAnimation

`liberadeltrajo.es.chaosarena2.entity`

Sistema de animación por frames que sustituye TextureAtlas + Animation de libGDX. Carga frames PNG individuales desde assets/, los ordena alfabéticamente y los reproduce a velocidad configurable. La convención de nomenclatura es: [personaje]_[acción]_NNN.png (ej: liu_kang_idle_001.png, liu_kang_idle_002.png, ...).

Enums

```
enum { PlayMode: LOOP, NORMAL }
```

-> LOOP: el índice vuelve a 0 al superar el último frame (módulo). NORMAL: se clipa en el último frame e `isFinished()` devuelve true.

Campos

```
Bitmap[] frames
```

-> Array de Bitmaps ordenados por nombre de fichero. Todos los frames de la animación.

```
float frameDuration
```

-> Segundos por frame. Ej: 0.08 -> 12.5 FPS. 0.12 -> ~8.3 FPS.

```
PlayMode playMode
```

-> Modo de reproducción asignado en el constructor.

```
float totalDuration = frameDuration * frames.length
```

-> Duración total de la animación. Usada por `isFinished()` y `Player.updateAttack()` para saber cuando termina un ataque.

Métodos de instancia

```
getKeyFrame(float stateTime) -> Bitmap
```

-> LOOP: `idx = (int)(stateTime / frameDuration) % frames.length`. NORMAL: `idx = Math.min((int)(stateTime / frameDuration), frames.length - 1)`.

```
isFinished(float stateTime) -> boolean
```

-> Solo tiene sentido en NORMAL. Devuelve `stateTime >= totalDuration`.

```
getDuration() -> float
```

-> Devuelve `totalDuration`.

```
getFrameWidth() / getFrameHeight() -> int
```

-> Dimensiones del primer frame. Devuelven 1 si el array está vacío (fallback). `Player.calibrateScale()` los usa para los cálculos de escala. `Player.loadAnimations()` los usa para detectar si la animación existe (>1).

```
recycle()
```

-> Llama a `Bitmap.recycle()` en cada frame no reciclado. Libera memoria nativa.

Métodos estáticos

```
load(AssetManager, folder, prefix, frameDuration, PlayMode) -> SpriteAnimation
```

-> Lista `assets.list(folder)`, filtra los que empiecen por 'prefix' y terminen en .png o .webp, ordena la lista, decodifica cada uno con `inScaled=false`. Si no encuentra frames devuelve `fallback()`.

```
fallback(float frameDuration, PlayMode mode) -> SpriteAnimation
```

-> Crea un Bitmap 1x1 blanco (0xFFFFFFFF, ARGB_8888). Player detecta esta situación comprobando `getFrameWidth() == 1`.

Paquete: riberadeltajo.es.chaosarena2.ai**class EnemyAI**

riberadeltajo.es.chaosarena2.ai

Controlador de IA del enemigo basado en maquina de estados finita de 5 estados. Toma decisiones basadas en la distancia al jugador, el cooldown de ataque y las acciones del jugador. Incluye comportamiento reactivo: si el jugador usa SPECIAL, la IA retrocede automaticamente. El tipo de ataque elegido depende de la distancia y del nivel de comboCharge del NPC.

```
enum { State: IDLE, CHASE, ATTACK, RETREAT, COOLDOWN }
```

-> IDLE: espera hasta que stateTimer expire y luego llama a decideNextState(). CHASE: se mueve hacia el objetivo hasta llegar a rango de ataque (<250px). ATTACK: ejecuta executeProAttack() y pasa a COOLDOWN. RETREAT: se aleja del objetivo durante stateTimer segundos. COOLDOWN: pausa post-ataque; puede transicionar a RETREAT aleatoriamente.

Campos

```
Player npc
```

-> El Player que controla la IA (player2 en Juego). La IA llama a npc.move() y npc.attack().

```
Player target
```

-> El Player objetivo (player1, el humano). La IA lee target.x y target.isAttacking().

```
float stateTimer
```

-> Tiempo restante en el estado actual en segundos. Se decrementa cada frame. Cuando llega a 0 se reevalua la logica del estado.

```
float attackCooldownTimer
```

-> Tiempo de espera entre ataques. Se fija a attackRate tras cada ataque. La IA no puede pasar a ATTACK mientras este > 0.

```
float speed = 350
```

-> Velocidad de movimiento del NPC en px/s. Se multiplica por delta en npc.move(). Configurable por RunManager y applyModeConfig() en Juego.

```
float damage = 5
```

-> Valor de referencia del dano. setDamage(v) tambien ajusta npc.damageMultiplier = v/10f para que el escalado sea consistente con el sistema de stats del Player.

```
float attackRate = 1.8
```

-> Segundos base de cooldown entre ataques. El delay real es attackRate + random(-0.2, +0.4) para dar variabilidad.

Métodos públicos

```
update(float delta)
```

-> Si alguno tiene currentHealth<=0: retorna. Calcula dist=target.x-npc.x y absDist. Decrementa timers. Actualiza npc.facingRight si no esta atacando. Si npc.isHurt(): IDLE, stateTimer=0.2s. Comportamiento reactivo: si target usa SPECIAL -> RETREAT 0.5s; si target ataca, dist<250 y cooldown activo -> 70% prob de RETREAT 0.4s. Switch sobre currentState con la logica de cada estado.

```
setSpeed/setDamage/setAttackRate / getSpeed/getDamage/getAttackRate
```

-> Getters y setters. setDamage actualiza tambien npc.damageMultiplier = v/10f.

Métodos privados

```
decideNextState(float absDist)
```

-> Transicion segun distancia: absDist > 350 -> CHASE (stateTimer=0.2..0.6s). absDist < 180 -> 30% RETREAT (0.2..0.4s), sino ATTACK si !cooldown, sino IDLE (0.1s). 180..350 -> si !cooldown: 80% ATTACK, 20% CHASE; si cooldown: 40% RETREAT, 60% IDLE.

```
executeProAttack(float distance)
```

-> Elige tipo: SPECIAL si comboCharge>=MAX_COMBO_CHARGE; KICK si distance>200; sino PUNCH(60%) o KICK(40%). Llama a npc.attack(type). Fija stateTimer y attackCooldownTimer = attackRate + random en [-0.2, 0.4].

Paquete: riberadeltajo.es.chaosarena2.input**class VirtualJoystick**

riberadeltajo.es.chaosarena2.input

Joystick táctil dibujado en Canvas que sustituye el touchpad de libGDX Scene2D. Soporta multi-touch rastreando un único puntero activo (trackingPointerId). Requiere una Matrix inversa para convertir coordenadas de pantalla a coordenadas mundo. Juego usa getKnobPercentX() con umbral ± 0.2 para detectar movimiento horizontal.

Campos

float centerX, centerY

-> Centro del joystick en coordenadas mundo. Fijado en (280, WORLD_H-280 = 800). No cambia durante la partida.

float radius = 180

-> Radio de la zona activa de la base en px mundo. El área de detección de toque tiene un margen de 1.3x (234px) para facilitar el uso con el pulgar.

float knobRadius = radius * 0.70 = 126

-> Radio con el que se dibuja la bola del joystick. La bola no puede alejarse más de 'radius' del centro (clipping en updateKnob).

float knobX, knobY

-> Posición actual del centro de la bola. En reposo = (centerX, centerY). Se actualiza cada frame de ACTION_MOVE.

int trackingPointerId

-> -1 si libre. ID del puntero que controla el joystick si está activo. Permite ignorar otros dedos en la pantalla.

Matrix inverseMatrix

-> Inversa de worldMatrix. Inyectada por Juego.updateScaleMatrix(). Convierte puntos de toque de px pantalla a coordenadas mundo antes de comparar.

Métodos

onTouchEvent(MotionEvent e) -> boolean

-> ACTION_DOWN/POINTER_DOWN: si el toque cae en el área de la base (radio*1.3) y trackingPointerId==-1, registra el puntero y llama a updateKnob(). ACTION_MOVE: si hay puntero activo, busca su índice actual y actualiza la bola. ACTION_UP/POINTER_UP/CANCEL: si es el puntero activo, lo libera y centra la bola.

updateKnob(float tx, float ty)

-> Calcula $dx=tx-centerX$, $dy=ty-centerY$. Si $\sqrt{dx^2+dy^2} > radius$: normaliza (dx,dy) al radio (clipping circular). $knobX = centerX + dx$; $knobY = centerY + dy$.

getKnobPercentX() -> float

-> $(knobX - centerX) / radius$. Rango exacto [-1.0, +1.0]. Juego lee: si > 0.2 mueve derecha, si < -0.2 mueve izquierda.

getKnobPercentY() -> float

-> $(knobY - centerY) / radius$. No se usa actualmente en la lógica de combate.

isActive() -> boolean

-> trackingPointerId != -1. Indica si hay un dedo activo en el joystick.

draw(Canvas canvas)

-> Dibuja bgBitmap en RectF(center-radius, center+radius) y knobBitmap en RectF(knob-knobRadius, knob+knobRadius) con FILTER_BITMAP_FLAG.

Paquete: riberadeltajo.es.chaosarena2.stage**class StageDef**

riberadeltajo.es.chaosarena2.stage

Contenedor de datos de un escenario de combate. Encapsula el fondo visual, los parametros de posicion del suelo y un mapa opcional de offsets Y por personaje para ajuste fino. Se instancian 3 escenarios en ResourceManager y se selecciona uno en la pantalla de seleccion de personaje.

Campos publicos

String name

-> Nombre del escenario mostrado en la pantalla CHAR_SELECT. Valores: 'Bosque', 'Desierto', 'Ciudad'.

Bitmap bgBitmap

-> Imagen de fondo del escenario cargada desde assets/backgrounds/. Se dibuja en drawFighting() con desplazamiento parallax horizontal.

float floorVisualOffset

-> Offset visual del suelo respecto a la base de la imagen. Permite ajustar visualmente donde parece estar el suelo en el arte. Bosque=-40, Desierto=0, Ciudad=+40.

float groundY

-> Coordenada Y mundo donde se apoyan los personajes. Bosque=80, Desierto=60, Ciudad=90. Se pasa al constructor de Player como parametro 'y'.

Metodos

StageDef(name, bgBitmap, floorVisualOffset, groundY)

-> Constructor que asigna los cuatro campos publicos.

addOffset(String charName, float offset) -> StageDef

-> Añade un offset de Y en el mapa charYOffsets para un personaje especifico. Retorna this para encadenado de llamadas. No se usa actualmente en la creacion de escenarios en ResourceManager.

getOffsetFor(String charName) -> float

-> Busca el offset en charYOffsets. Si no existe devuelve 0f. Juego lo usa en startFight(): player.y = groundY + stage.getOffsetFor(playerName).

recycle()

-> Llama a bgBitmap.recycle() si el Bitmap no esta ya reciclado. Llamado desde ResourceManager.recycle() en el onDestroy de la actividad.

Paquete: riberadeltajo.es.chaosarena2.progression**class Buff**[riberadeltajo.es.chaosarena2.progression](#)

Representa una mejora permanente para el jugador en modo Arcade. Encapsula tipo, valor, permanencia y rareza. Se generan en grupos de 3 por RunManager y se ofrecen al jugador; los no elegidos se aplican al enemigo siguiente.

```
enum { Rarity: COMMON, RARE, EPIC }
```

-> COMMON (prob 40%): mult=1.0. RARE (prob 30%): mult=1.5. EPIC (prob 10%): mult=2.5. La rareza determina el color del boton en BUFF_SELECT: EPIC=naranja oscuro, RARE=cyan oscuro, COMMON=gris.

Campos y métodos

BuffType type

-> Tipo de stat que mejora (ver BuffType).

float value

-> Valor del efecto ya multiplicado por rareza. DAMAGE: 0.15/0.225/0.375. SPEED: 0.10/0.15/0.25. HEALTH: 30/45/75. LIFESTEAL: 0.03/0.045/0.075. ARMOR: 0.05/0.075/0.125. CRITICAL: 0.10/0.15/0.25. REGEN: 2/3/5 hp/s.

boolean isPermanent

-> Siempre true. Reservado para posibles buffs temporales en versiones futuras.

Rarity rarity

-> Rareza del buff. Determina color de fondo del boton de seleccion.

getDescription() -> String

-> Retorna '[RARITY] +NombreStat'. Ej: '[EPIC] +Dano', '[COMMON] +Regen. Vida'.

enum BuffType[riberadeltajo.es.chaosarena2.progression](#)

Enumera los 7 stats mejorables. Usado como llave en los switch de RunManager y Buff.

enum { DAMAGE }

-> Aumenta Player.damageMultiplier.

enum { SPEED }

-> Aumenta Player.speedMultiplier.

enum { HEALTH }

-> Aumenta Player.maxHealth y currentHealth.

enum { LIFESTEAL }

-> Aumenta Player.lifestealPercent.

enum { ARMOR }

-> Aumenta Player.armorPercent (reduccion de dano recibido).

enum { CRITICAL }

-> Aumenta Player.criticalChance (probabilidad de golpe x2).

enum { REGEN }

-> Aumenta Player.regenPerSec (regeneracion pasiva de vida).

class Event[riberadeltajo.es.chaosarena2.progression](#)

Evento especial que ocurre cada 3 victorias en Arcade (shouldTriggerEvent). Algunos benefician al jugador (HEAL) y otros penalizan o modifican el siguiente combate.

```
enum { EventType: HEAL, HARDER_COMBAT, GLOBAL_MODIFIER, POISON, ENEMY_HEAL }
```

-> HEAL: `currentHealth += maxHealth*0.3` (clampeado a `maxHealth`). HARDER_COMBAT: `RunManager.harderCombatNext=true` (enemigo +50% vida/dano/velocidad). GLOBAL_MODIFIER: `globalDamageMultiplier=1.5` (dano de ambos x1.5 en la próxima pelea). POISON: `currentHealth -= maxHealth*0.2` (mínimo 1). ENEMY_HEAL: `enemyHealNext=true` (próximo enemigo +100 vida máxima).

Campo `apply()`

`EventType type`

-> Tipo del evento. Determina que efecto aplica `apply()`.

`String description`

-> Texto mostrado al jugador en la pantalla EVENT antes de pulsar continuar.

`apply(RunManager runManager)`

-> Ejecuta el efecto: HEAL y POISON modifican `currentHealth` directamente via `runManager.getPlayer()`. El resto llaman a setters de `RunManager`.

class RunManager

fiberadeltajo.es.chaosarena2.progression

Gestor de estado de una partida completa en modo Arcade. Es la única fuente de verdad sobre el estado de la 'run': buffs activos, modificadores del próximo combate y escalado progresivo de enemigos. Instanciado por Juego al inicio de cada partida Arcade y destruido implícitamente al volver al menú principal.

Constantes y campos

static final int MAX_BUFF_SLOTS = 6

-> Límite de buffs acumulables. applyBuffToPlayer() ignora buffs adicionales si se supera.

Player player

-> Referencia al jugador. Puede actualizarse con setPlayer() si el objeto se recrea. reapplyBuffs() re-aplica todos los buffs activos al nuevo objeto.

int fightsWon

-> Contador de combates ganados en la run. Determina el escalado progresivo: >=3 -> 1 buff extra al enemigo; >=6 -> 2 buffs extra al enemigo.

List<Buff> activeBuffs

-> Buffs elegidos por el jugador. Máximo MAX_BUFF_SLOTS=6.

boolean harderCombatNext

-> Si true: próximo enemigo tiene vida/dano x1.5 y velocidad x1.2. Se resetea en onFightWon().

boolean enemyHealNext

-> Si true: próximo enemigo recibe +100 vida máxima adicional. Se resetea en onFightWon().

float globalDamageMultiplier

-> Multiplicador de dano aplicado a ambos jugadores el próximo combate. Normalmente 1.0; sube a 1.5 con evento GLOBAL_MODIFIER. Se resetea en onFightWon().

List<Buff> pendingEnemyBuffs

-> Buffs que el jugador rechaza en BUFF_SELECT. Se aplican al siguiente enemigo en applyEnemyScaling() y se limpian.

Métodos de progresión

onFightWon()

-> fightsWon++. Resetea harderCombatNext=false, enemyHealNext=false, globalDamageMultiplier=1.0. Llamado en Juego.showResult() tras victoria Arcade.

shouldTriggerEvent() -> boolean

-> fightsWon > 0 && fightsWon % 3 == 0. Cada 3 victorias hay evento especial.

generateRandomEvent() -> Event

-> r = random(0..4). Selecciona uno de los 5 EventType con probabilidad uniforme.

Gestión de buffs del jugador

getRandomBuffChoices() -> List<Buff>

-> Genera 3 Buff. Para cada uno: BuffType aleatorio. r > 0.9 -> EPIC (mult=2.5); r > 0.6 -> RARE (mult=1.5); sino COMMON (mult=1.0). Valor base por tipo multiplicado por mult de rareza.

applyBuffToPlayer(Buff buff)

-> Si activeBuffs.size() < MAX_BUFF_SLOTS: anade a la lista y llama a applyBuffDirectly().

addPendingEnemyBuff(Buff buff)

-> Anade el buff rechazado a pendingEnemyBuffs. Llamado por Juego.handleTap() para los buffs no elegidos.

Escalado de enemigos

applyEnemyScaling(EnemyAI enemyAI, Player enemyPlayer)

-> Aplica en orden: (1) harderCombatNext: vida/dano*1.5, velocidad*1.2; (2) enemyHealNext: maxHealth+=100, currentHealth=maxHealth; (3) globalDamageMultiplier al dano y damageMultiplier del enemy; (4) pendingEnemyBuffs -> applyEnemyBuff() para cada uno, luego clear(); (5) escalado progresivo: fightsWon>=3 -> 1 buff random; fightsWon>=6 -> 2 buffs random.

applyEnemyBuff(EnemyAI, Player, BuffType)

-> Aplica buff con valores fijos de enemigo: DAMAGE: damageMultiplier+=0.2. SPEED: speed*1.15. HEALTH: maxHealth+=50. LIFESTEAL: lifestealPercent+=0.05. ARMOR: armorPercent+=0.10. CRITICAL: criticalChance+=0.15. REGEN: regenPerSec+=2.

Dependencias entre clases

ActividadJuego	-> engine.Juego, engine.ResourceManager
engine.Juego	-> engine.BucleJuego, engine.ResourceManager, entity.Player, ai.EnemyAI, input.VirtualJoystick, stage.StageDef, progression.Buff, progression.Event, progression.RunManager
engine.ResourceManager	-> stage.StageDef
entity.Player	-> entity.SpriteAnimation
ai.EnemyAI	-> entity.Player
progression.Event	-> entity.Player, progression.RunManager (mismo paquete)
progression.RunManager	-> entity.Player, ai.EnemyAI, progression.Buff, progression.BuffType
input.VirtualJoystick	-> (solo Android SDK)
stage.StageDef	-> (solo Android SDK)
entity.SpriteAnimation	-> (solo Android SDK)
progression.Buff	-> progression.BuffType
progression.BuffType	-> (enum, sin dependencias)

Flujo de ejecución - Modo Arcade

1. ActividadJuego.onCreate
 - > new ResourceManager(context): carga assets, fuentes, escenarios
 - > new Juego(activity, res, prefs): crea VirtualJoystick, inicializa botones
 - > setContentView(juego): la SurfaceView ocupa toda la pantalla
2. Juego.surfaceCreated: calcula worldMatrix, new BucleJuego.start(), startMusic()
3. BucleJuego.run() a 60 FPS:
 - > Juego.actualizar(delta=0.01667s) -> updateFighting(delta)
 - > Juego.renderizar(canvas) -> drawFighting(canvas)
4. Usuario toca 'ARCADE' -> currentScreen=CHAR_SELECT
 - > Elige personaje+escenario -> Juego.startFight()
 - new Player(jugador), new Player(enemigo), new EnemyAI, new RunManager
 - currentScreen=FIGHTING
5. Combat loop (updateFighting cada frame):
 - joystick.getKnobPercentX() -> player1.move() si $|x| > 0.2$
 - player1.update/updateAttack(delta) | player2.update/updateAttack(delta)
 - enemyAI.update(delta): maquina estados -> npc.move() o npc.attack()
 - Si justHitThisFrame: triggerHitFlash (circulo amarillo 100ms)
 - Si currentHealth<=0: triggerResult(won)
6. triggerResult: pendingResult=true, startDeath/forceIdle
 - updateFighting acumula deathDelay hasta 2.5s -> showResult()
 - showResult: currentScreen=RESULT, runManager.onFightWon(), enemiesDefeated++
7. 'ELEGIR MEJORA' -> BUFF_SELECT: runManager.getRandomBuffChoices() -> 3 Buff
 - Jugador elige: runManager.applyBuffToPlayer(elegido)

Rechazados: `runManager.addPendingEnemyBuff(rechazado) x2`

`checkForEvent: fightsWon%3==0 -> EVENT screen, Event.apply(runManager)`

`resetCombat: spawnNewEnemy + runManager.applyEnemyScaling(enemyAI, player2)`

8. Derrota (`currentHealth<=0`) -> `saveArcadeScore()` -> top-5 en `SharedPreferences`

`currentScreen=MAIN_MENU`